CSE 144 Final Project Report

Transfer Learning Challenge

Group Members:
Allexindir Smith (CruzID: asmith67@ucsc.edu)

Spring 2026

1 Introduction

Problem goal and setting. The task is a 100-class image classification problem with an extremely low ammount of data: only about 10 training images per class (1,081 labeled images total), with 1036 unlabeled test images to predict. This is effectively a few-shot learning problem. With so few examples per class, training a deep network from scratch is infeasible as it would immediately memorize the training set and fail to generalize. The objective is to maximize test set accuracy on Kaggle, where the baseline to beat is 60% accuracy.

Why transfer learning is appropriate. With roughly 10 images per class, the only viable path to strong accuracy is to reuse representations learned on large external datasets. Modern self-supervised and contrastive vision backbones (DINOv3, SigLIP2) are trained on hundreds of millions to over a billion images and produce general-purpose features that linearly separate a wide range of object categories. By freezing such a backbone and training only a lightweight classification head, we transfer this prior knowledge into our 100-class problem while keeping the number of trainable parameters small enough to avoid overfitting on 1000 images.

Approach and main result. We use a frozen pretrained vision backbone (DINOv3 ViT-L/16) as a fixed feature extractor, with a lightweight trainable head (linear probe and small MLP were both tried). We then adapt the last four transformer blocks of the DINOv3 backbone with low-rank adaptation (LoRA). We apply heavy data augmentation during training and 5-way test-time augmentation (TTA) at inference. With just DINOv3 + LoRA we reach about 90% validation accuracy comfortably above the 60% baseline. However to push this further we additionally trained a SigLIP backbone which gave around a 92% validation accuracy, ensembling both models at test time gave a **93%** accuracy.

2 Dataset

Sizes and classes. The dataset has 100 classes. There are 1081 labeled training images and 1036 unlabeled test images. We hold out a small validation set with a fixed number of images per class for model selection. The default is 2 validation images per class, giving a 200 image validation set and roughly 879 images for training.

Split	Images
Train (after split)	879
Validation	200
Test (unlabeled)	1036
Total labeled	1081

Table 1: Dataset sizes. Validation uses 2 stratified images per class.

Directory structure and label mapping. Training images are organized into per-class directories named "0", "1", . . . , "99". The directory name is the integer label. Test images are stored as "0.jpg", "1.jpg", . . . , sorted by integer ID. A critical correctness detail is that the class directories must be sorted numerically, not as strings. Python's default string sort yields "0", "1", "10", "11", . . . , "2", "20", . . . , which scrambles labels and caps accuracy near 1%. Our dataset class sorts with sorted(names, key=int), maps each directory name directly to int(name), and asserts the mapping on initialization.

Preprocessing and augmentation. Images are resized to 256×256 for the DINOv3 backbone and to the SigLIP2 processor's native 384×384 for that backbone, then normalized. DINOv3 uses ImageNet normalization (mean $(0.485, 0.456, 0.406)$, std $(0.229, 0.224, 0.225)$); SigLIP2 uses the mean/std reported by its own image processor. Training augmentation is intentionally heavy to combat the tiny training set:

- `RandomResizedCrop` with scale $(0.6, 1.0)$
- `RandomHorizontalFlip`
- `RandAugment` (`num_ops=2`, `magnitude=9`)
- `ColorJitter` (brightness 0.2, contrast 0.2, saturation 0.1, hue 0.05)
- `RandomErasing` with $p = 0.25$

The SigLIP2 path uses a lighter augmentation (`RandomResizedCrop` scale $(0.7, 1.0)$ plus horizontal flip) matched to its processor normalization. Validation and test use a deterministic resize-and-normalize transform.

3 Implementation

3.1 Model

Pretrained backbone and rationale. We evaluate two frozen backbones behind a common wrapper interface:

- **DINOv3 ViT-L/16** (`vit_large_patch16_dinov3.lvd1689m` via timm), a self-supervised Vision Transformer whose CLS-token features are known to be strong linear-probe representations.
- **SigLIP2 ViT-so400m/14-384** (`google/siglip2-so400m-patch14-384` via transformers), a sigmoid-loss contrastive image text model whose pooled image embedding is highly discriminative.

These were chosen because large self-supervised, contrastive ViTs provide the most transferable features available off the shelf, which matters most when only 10 labels per class are available.

Architecture changes. We discard the backbone's original classification layer and attach a lightweight trainable head on top of the pooled feature vector:

- **Linear head**: a single `Linear(D, 100)` layer.
- **MLP head**: Linear(D, 512) → GELU → Dropout → Linear(512, 100), with dropout 0.2 (DINOv3) or 0.4 (SigLIP2). Head weights are initialized with a truncated-normal (std = 0.02) and zero bias.

Fine-tuning strategy. The backbone is frozen by default. Full fine-tuning is deliberately *not* supported. with only 1000 training images we risk overfitting catastrophically. The only optional backbone adaptation is **LoRA** on the last 4 transformer blocks: low-rank adapters ($r = 8$, $\alpha = 16$, $lora\_dropout = 0.05$) are attached to the attention projections via the peft library, and all non-LoRA backbone parameters remain frozen. In the LoRA configuration the model has 303,786,596 total parameters but only 707,172 trainable (131,072 LoRA parameters plus the MLP head), which keeps capacity in check.

3.2 Training

Loss and optimizer. We minimize cross-entropy with label smoothing of 0.1, optimized with AdamW. Parameters are split into groups: the head trains at learning rate 10^{-3} and the LoRA parameters train at 10^{-4}, both with weight decay 0.01.

Hyperparameter	Value
Epochs	50
Batch size	32
Image size	256 (DINOv3) / 384 (SigLIP2)
Head learning rate	1×10^{-3}
Backbone (LoRA) learning rate	1×10^{-4}
Weight decay	0.01
LR schedule	Linear warmup (5 ep.) → cosine anneal
Label smoothing	0.1
Seed	42

Table 2: Training configuration.

Schedule and other hyperparameters. The learning-rate schedule is a 5-epoch linear warmup (start factor 0.1) followed by cosine annealing over the remaining epochs.

Hardware/software environment. Training was performed on an Apple Silicon MacBook (M5 Pro) using the PyTorch **MPS** backend. Key software: Python 3.11, PyTorch 2.8.0, timm (≥1.0.20), transformers (≥4.45), peft (≥0.7), and OmegaConf for configuration. Data loading uses `num_workers=0` because macOS multiprocessing with MPS is unreliable. On MPS it is disabled to avoid Metal driver instability.

4 Experiments

Baseline setup. The baseline was a **linear probe** on frozen DINOv3 features: a single linear layer trained on top of the fixed embedding, with the augmentation and schedule described above.

Hyperparameter tuning and validation method. Model selection uses a stratified validation set of 2 images per class (200 images total), constructed by shuffling each class's indices with a fixed seed and reserving the first two. We track validation accuracy every epoch, write per-epoch metrics to JSON, and keep the checkpoint with the best validation accuracy. There was not much hyperparameter tuning done as most of my time was spent building the large scale changes, however I did run a few different values for various hyperparameters and didnt notice a huge change in performance.

Ablations. We ran three end-to-end configurations that ablate backbone choice, head complexity, and backbone adaptation:

1. **DINOv3 + linear head** (frozen): the linear-probe baseline.
2. **DINOv3 + MLP head + LoRA** (last 4 blocks): adds head capacity and limited backbone adaptation.
3. **SigLIP2 + MLP head** (frozen): swaps the backbone to a larger contrastive ViT.

Heavy augmentation (RandAugment, RandomErasing, color jitter) and 5-way TTA are applied consistently. The augmentation stack is essential: without it, the head fits the 880 training images within a few epochs (training accuracy reaches 100%) and validation accuracy degrades.

5 Results

Training/validation accuracy. Table 3 reports the best validation accuracy reached by each configuration (with the epoch at which it occurred). All three models reach ∼99–100% training accuracy, reflecting the tiny training set. Validation accuracy is the meaningful metric. The optional loss/accuracy curves produced by scripts/plot_curves.py are saved under outputs/plots/.

Configuration	Best val acc.	Epoch	Train acc.
DINOv3 ViT-L/16, linear probe (frozen)	88.5%	33	99.1%
DINOv3 ViT-L/16, MLP head + LoRA (last 4 blocks)	90.0%	29	99.4%
SigLIP2 so400m/14-384, MLP head (frozen)	**92.0%**	9	98.4%

Table 3: Validation accuracy by configuration. Each model's best-validation checkpoint is saved separately.

Kaggle public leaderboard score. 93.6% (rank #8). Submissions are generated with 5-way TTA (averaged softmax over a direct resize, a horizontal flip, and three rescaled center crops) and written to `submission.csv` with columns `ID,Label`, one row per test image, sorted by integer ID. An optional weighted-probability ensemble of the DINOv3 and SigLIP2 models is also supported.

6 Discussion

What worked best and why. The best approach was the ensemble. Heavy augmentation followed by DINOv3 + LoRA ensembled with SigLIP2 ending in TTA gave the best results overall. Although all the small features I added gave small improvements the most prominent architecture are the vision transformers which can generalize basic patterns very well.

Failure cases, overfitting/underfitting. Overfitting is the dominant risk: every configuration reaches ∼100% training accuracy well before the validation accuracy peaks. Label smoothing, weight decay, dropout, heavy augmentation, and best-checkpoint selection on validation accuracy are the countermeasures. We deliberately avoided full backbone fine-tuning in order to help aleviate this. Most failure cases that were encountered were due to matrix sizing, and the names of the test and train data.

Limitations and next improvements. The validation set is only 200 images, so reported accuracies carry ∼1% noise. Cross-validation across multiple stratified splits would tighten the estimates. Next steps: (1) light LoRA on the SigLIP2 backbone. (2) K-fold cross-validation.

7 Reproducibility

Seeds and determinism. A single set_seed utility seeds torch, numpy, random, and PYTHONHASHSEED and enables torch.use_deterministic_algorithms(True, warn_only=True). Each checkpoint stores the resolved config, seed, epoch, and best validation accuracy. However there seems to be some difference in output depending on weather MPS or CUDA is used. I am not sure how to aleviate this.

Package versions / environment. Python 3.11 on Apple Silicon (MPS). Install dependencies with:

```
pip install -r requirements.txt
```

Exact commands.

```
# Train DINOv3 + LoRA + MLP
PYTORCH_ENABLE_MPS_FALLBACK=1 \
  python scripts/train.py --config configs/dinov3_linear.yaml

# Train SigLIP2
PYTORCH_ENABLE_MPS_FALLBACK=1 \
  python scripts/train.py --config configs/siglip_linear.yaml

# Generate submission.csv
python scripts/predict.py \
  --checkpoints outputs/checkpoints/siglip_linear_best.pth outputs/checkpoints/
    dinov3_lora_best.pth \
  --weights 1.0 1.0 \
  --out submission.csv
```

8 Team Contributions

I was the only member who worked on this project so I will skip this section.

9 References

1. PyTorch and TorchVision documentation (transforms, RandAugment, RandomErasing). `https://pytorch.org/vision/stable/transforms.html`
2. Siméoni et al. *DINOv3*. arXiv:2508.10104. `https://arxiv.org/abs/2508.10104`
3. Tschannen et al. *SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features*. arXiv:2502.14786. `https://arxiv.org/abs/2502.14786`
4. Facebook Research. *DINOv3* (official code and weights). `https://github.com/facebookresearch/dinov3`
5. Hugging Face Transformers documentation. *SigLIP2 model*. `https://huggingface.co/docs/transformers/en/model_doc/siglip2`
6. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685. `https://arxiv.org/pdf/2106.09685`